

Machine Learning | Assignment 2 | Aditya Mallakula

1. Exploratory Data Analysis (40 points)

- a. Consider the following numeric variables in the dataset: Mean Radius, Mean Texture, Mean Perimeter, Mean Area, Mean Smoothness, Mean Compactness, Mean Concavity and Mean Concave Points. Summarize the statistics of these variables into count, mean, standard deviation, minimum, 25% percentile, 50% percentile, 75% percentile, and maximum.

I used the “describe()” function, present in pandas library, to illustrate the count, mean, standard deviation, minimum value, 25% percentile, 50% percentile, 75% percentile and maximum value for each of the above mentioned features.

	Mean Radius	Mean Texture	Mean Perimeter	Mean Area	Mean Smoothness	Mean Compactness	Mean Concavity	Mean Concave Points
count	198.000000	198.00000	198.000000	198.000000	198.000000	198.000000	198.000000	198.000000
mean	17.412323	22.27601	114.856566	970.040909	0.102681	0.142648	0.156243	0.086776
std	3.161676	4.29829	21.383402	352.149215	0.012522	0.049898	0.070572	0.033877
min	10.950000	10.38000	71.900000	361.600000	0.074970	0.046050	0.023980	0.020310
25%	15.052500	19.41250	98.160000	702.525000	0.093900	0.110200	0.106850	0.063670
50%	17.290000	21.75000	113.700000	929.100000	0.101900	0.131750	0.151350	0.086075
75%	19.580000	24.65500	129.650000	1193.500000	0.110975	0.172200	0.200500	0.103925
max	27.220000	39.28000	182.100000	2250.000000	0.144700	0.311400	0.426800	0.201200

My Observation:

From the output we can clearly state that there are no missing values in all the requested columns. Since Mean Area & Mean Perimeter have larger values (in the 100s), the mean, Standard deviation, 25% percentile, 50% percentile, 75% percentile are substantially larger compared to the other columns, which have lower or floating values.

- b. Consider the categorical variable “Outcome” in the dataset. Summarize the statistics of variables into count, unique value, top value, and frequency of top value.

I have used the “describe()” function, present in pandas library, to illustrate the count, unique, top value and frequency of the “Outcome” variable in the dataset. Since the values in Outcome are categorical, the output I got is:

```
count      198
unique       2
top         N
freq       151
Name: Outcome, dtype: object
```

My Observation:

Of the 198 values in the ‘Outcome’ column there are 2 unique values N & R. Of these, N is the most frequent value with 151 repetitions.

c. Is there a way to encode the Outcome variable from categorical to numerical data type? If so, how would you do that?

I used the “replace()” function present in pandas library to change the categorical data into numerical data. Using this function, I changed ‘N’ & ‘R’ as ‘1’ & ‘0’ respectively.

```
y.replace(['N', 'R'], [1, 0], inplace = True)
y.head()
```

```
0    1
1    1
2    1
3    1
4    0
Name: Outcome, dtype: int64
```

My Observation:

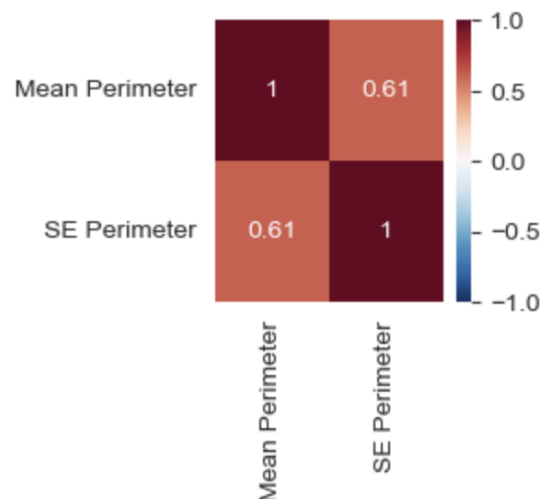
Since we cannot compute the model using non numeric values, we modified the categorical data into a numerical component. With this change we can now compute different parameters and derive numerical results.

d. What is the correlation between Mean Perimeter and SE Perimeter?

I used the “corr()” function from the pandas library to find the correlation between Mean Perimeter & SE Perimeter variables.

My Observation:

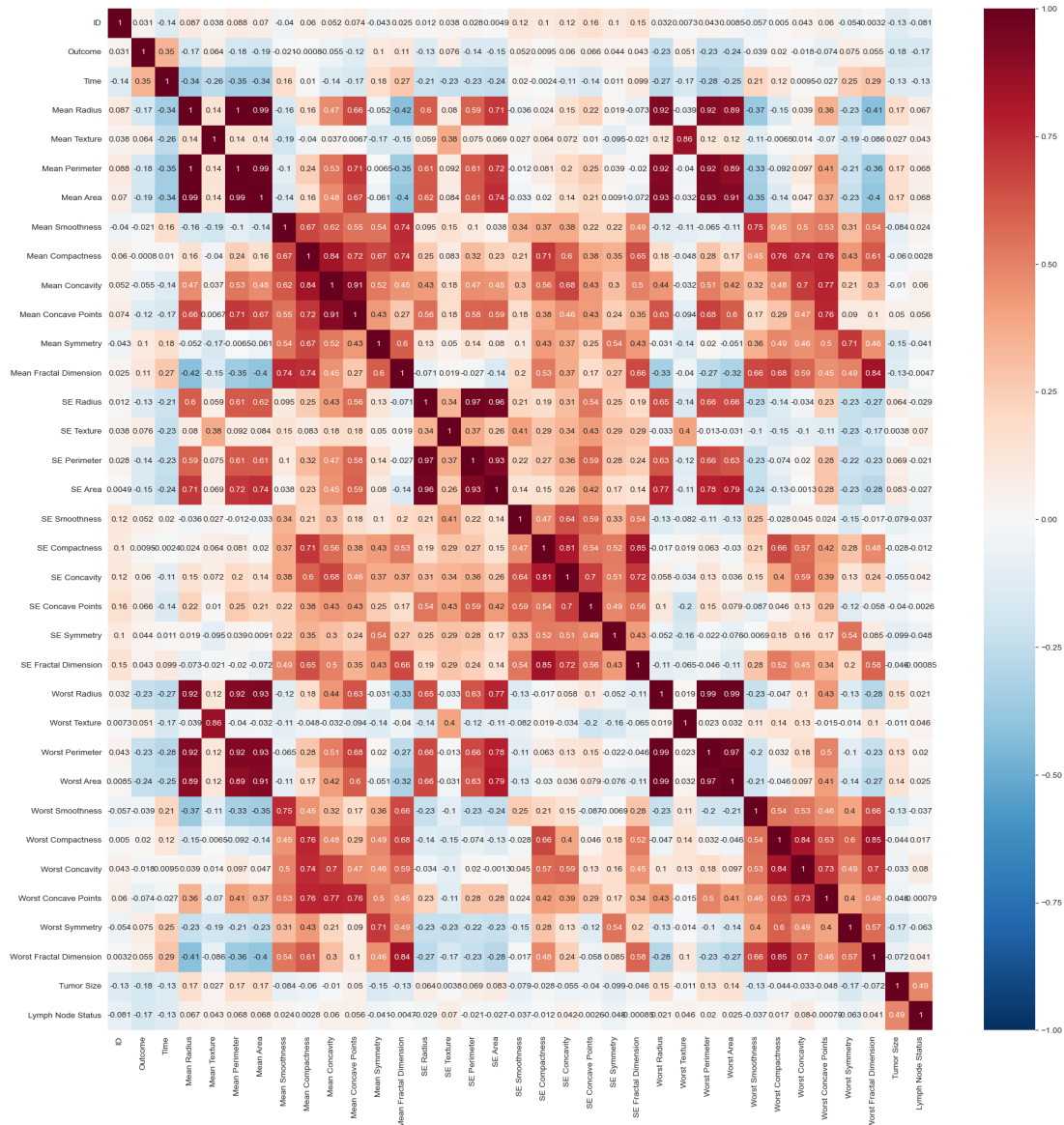
The correlation between these variables is approximately 0.61 (0.6099643781634987 to be exact), which is closer to 1.0. Based on this I believe the variables have a good correlation.



e. Do you think there are any redundant features present in the dataset? If so, explain how removing it won't impact the analysis. Also, based on the experiments so far, were there any interesting observations with respect to the variables?

Yes. I feel the dataset has some redundant features. I created a heat map that depicts the correlations between all the features in the dataset. At a glance there seem to be four sets of features that have a really good correlation with each other. They are as follows:

Feature Sets	Correlation
Mean Area & Mean Perimeter	0.99
Mean Concavity & Mean Concave Points	0.91
SE Perimeter & SE Area	0.93
Worst Perimeter & Worst Area	0.97



My Observation:

Based on the correlation, I feel that one of the features from each set can be removed as redundant features. From the above table, we can remove Mean Perimeter, Mean Concavity, SE Perimeter, Worst Perimeter columns and there would still be no difference in the model's performance. I feel that removing these features would increase the accuracy of the model.

Since the redundant features have been removed, the computation time required to train the model would also decrease drastically.

Apart from this, there are 3 variables that have a very bad relationship with all the other variables. They are Tumour Size, Lymph Node Status & Outcome. These three target variables do not have a dark area on the heat map.

2. Logistic Regression with One Variable (20 points)

a. Can you map the likelihood of breast cancer recurrence (Outcome) based on the "Mean Area" feature from the dataset?

I implemented logistic regression using "Outcome" as the target variable and "Mean Area" as the independent variable from the dataset. I split the data into training and test buckets with a 80:20 (Training : Test) ratio, set the learning rate to 0.7 and calculated theta values (Intercept and coefficient) using the Gradient Descent formula. I also used the sigmoid function to calculate the predictions.

The W & C (theta1 & theta0) values as calculated on the training data are:

- W (Theta1): -0.5114487368080947
- C (Theta0): 1.0185098298872004

The cost of this model as calculated on the testing data is:

- Cost: 0.45518989491955164

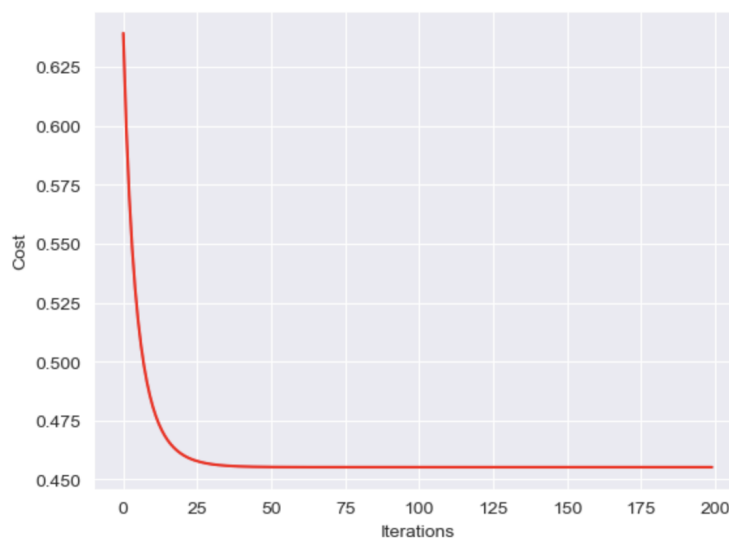


Fig 1: Graph shows the cost function being stabilised through multiple iterations

Once the predictions were derived, I rescaled them into 1s & 0s based on the factor that if the resultant prediction was ≥ 0.5 , it was 1 else it was changed to 0. Based on this we got a prediction set for Outcome.

b. Evaluate performance using a metric discussed in class (such as confusion matrix). You may also use graphs to explain your observations.

I compared the predictions from the model with the actual test Outcome data set and derived the confusion matrix. Based on the confusion matrix, I calculated the Precision, Recall, F1 Score & Accuracy.

- Confusion Matrix $[[TN, FP], [FN, TP]] = [[1, 4], [3, 32]]$
- Precision = 0.8888888888888888
- Recall = 0.9142857142857143
- F1 Score = 0.9014084507042254
- Accuracy = 0.825

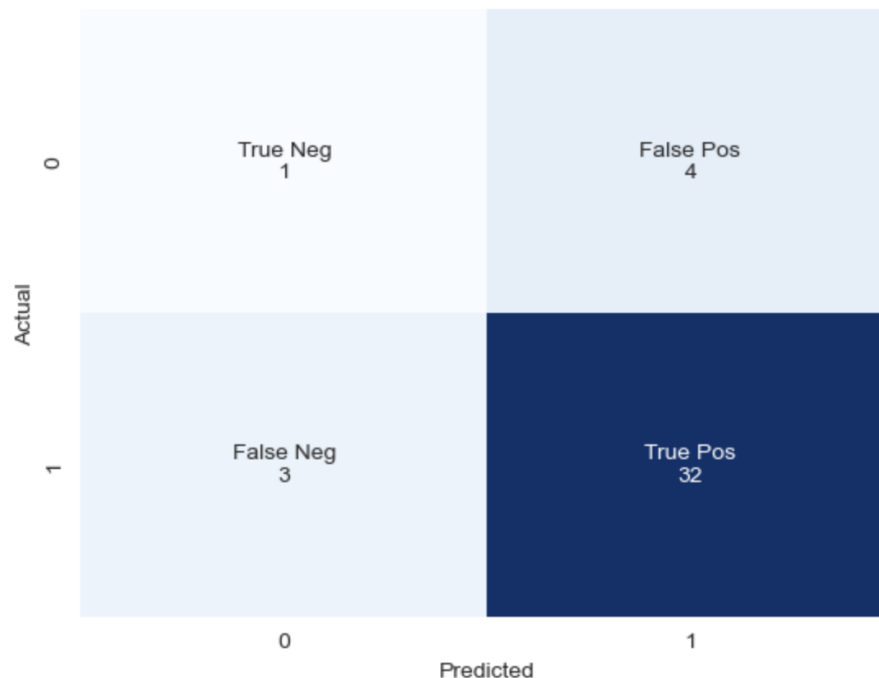


Fig 2: Confusion Matrix

My Observation:

The cost of the testing model is 0.455. The confusion matrix of the current model has a really good accuracy & f1 score of 0.825 & 0.901 respectively. The precision and recall are 0.889 & 0.914 respectively. Based on these results I feel that this is a good model.

3. Logistic Regression with Multiple Variables (50 points)

- Design a Logistic Regression model to predict breast cancer recurrence (Outcome) using the following 12 variables from the dataset as input features:**

Features: Mean radius, Mean texture, Mean perimeter, Mean area, Mean smoothness, Mean compactness, Mean concavity, Mean concave points, Mean fractal dimension, SE perimeter, SE texture, SE area.

I implemented logistic regression using “Outcome” as the target variable and the “Features” mentioned above as the independent variables from the dataset. I split the data into training and test buckets with a 80:20 (Training : Test) ratio, set the learning rate to 0.7 and calculated theta values (Intercept and coefficient) using the Gradient Descent formula on training data and then calculated the cost on the testing data. I also used sigmoid to calculate the predictions of the model.

The W & C (theta1 & theta0) values are:

- W (Theta1): [0.22482908 -0.07406684 -0.17476886 -0.57819363 -0.77798834 -0.02958095 0.3198775 0.12639061 0.60330531 -0.93603959 0.27796962 0.58242612]
- C (Theta0): 1.0910284510707866

The cost of this model is:

- Cost: 0.44830971732780567

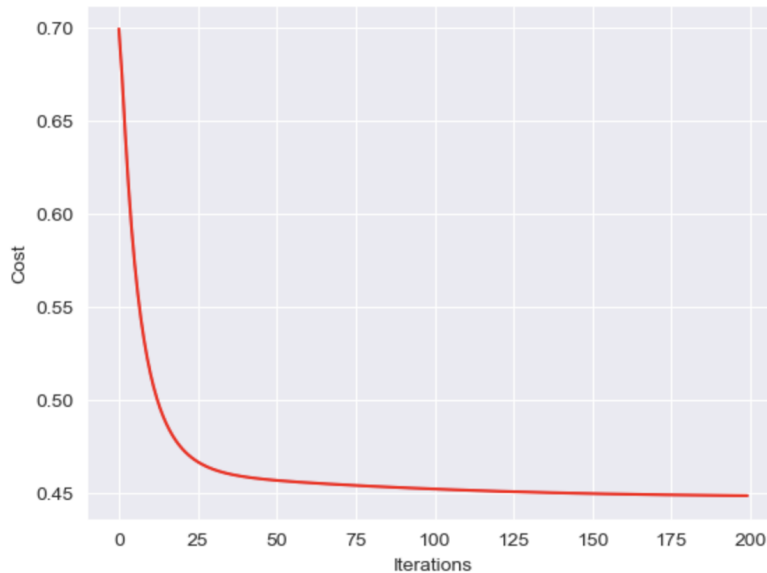


Fig 3: Graph shows the cost function being stabilised through multiple iterations
 Once the predictions were derived using the testing data, I rescaled them into 1s & 0s based on the factor that if the resultant prediction was ≥ 0.5 , the prediction would be 1 else it would be 0. Based on this we got a prediction set for Outcome.

The confusion matrix is as follows:

Performance Metric	Value
Confusion Matrix	[[2, 3], [4, 31]]
Precision	0.9117647058823529
Recall	0.8857142857142857
F1 Score	0.8985507246376812
Accuracy	0.825
Best Features	Mean radius, Mean texture, Mean perimeter, Mean area, Mean smoothness, Mean compactness, Mean concavity, Mean concave points, Mean fractal dimension, SE perimeter, SE texture, SE area

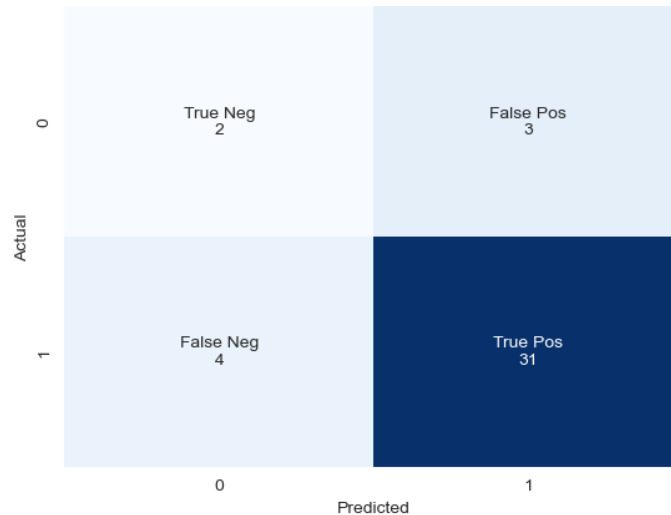


Fig 4: Confusion Matrix

My Observation:

Looking at the performance data, the Accuracy & F1 Score are 0.825 & 0.898 respectively. There are 31 true positives & 2 true negatives predicted out of the 40 values. Based on this, I can clearly say that the outcome predicted by the model is good.

- b. Design a Logistic Regression model to predict breast cancer recurrence (Outcome) using forward selection to select the most significant variables in the dataset as input features. Which subset of features gave you the best performance? What are your thoughts on these features getting selected? (Use 12 features from 3a as Input features).**

I have implemented Forward stepwise linear regression using the 12 features - mentioned above. I then split the data into training and test buckets in 80:20 (training : test) ratio and individually calculated the W & C (theta1 & theta0) values for each feature using the Gradient Descent formula on testing data.

Once the theta values were obtained, I then used those values to calculate the Confusion Matrix, Precision, Recall, F1 Score & Accuracy of the model. The feature with the best Accuracy & F1 Score values was merged with the rest of the remaining features and another iteration of the above was performed on the model that was obtained.

I continued this for a total of 12 iterations (since there were 12 columns) where I was able to get a final model with the best Accuracy & F1 Score values.

- The best model has the following performance values:

Performance Metric	Value
Confusion Matrix	[[0, 0], [5, 35]]
Precision	1.0
Recall	0.875
F1 Score	0.9333333333333333
Accuracy	0.875
Best Features	Mean Texture

Listed below are all the results for the best model:

- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension'}
- {'precision': 0.9428571428571428, 'recall': 0.868421052631579, 'accuracy': 0.825, 'f1': 0.904109589041096, 'confusion matrix': [[0, 2], [5, 33]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points'}
- {'precision': 0.9142857142857143, 'recall': 0.8648648648648649, 'accuracy': 0.8, 'f1': 0.8888888888888889, 'confusion matrix': [[0, 3], [5, 32]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture'}
- {'precision': 0.7714285714285715, 'recall': 0.9, 'accuracy': 0.725, 'f1': 0.8307692307692307, 'confusion matrix': [[2, 8], [3, 27]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter'}
- {'precision': 0.7142857142857143, 'recall': 0.8928571428571429, 'accuracy': 0.675, 'f1': 0.7936507936507937, 'confusion matrix': [[2, 10], [3, 25]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter, SE Area'}
- {'precision': 0.6571428571428571, 'recall': 0.8846153846153846, 'accuracy': 0.625, 'f1': 0.7540983606557377, 'confusion matrix': [[2, 12], [3, 23]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter, SE Area, Mean Radius'}
- {'precision': 0.6571428571428571, 'recall': 0.8846153846153846, 'accuracy': 0.625, 'f1': 0.7540983606557377, 'confusion matrix': [[2, 12], [3, 23]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter, SE Area, Mean Radius, Mean Perimeter'}
- {'precision': 0.6571428571428571, 'recall': 0.92, 'accuracy': 0.65, 'f1': 0.7666666666666667, 'confusion matrix': [[3, 12], [2, 23]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension,

Mean Concave Points, SE Texture, SE Perimeter, SE Area, Mean Radius, Mean Perimeter, Mean Area'}

My Observation:

If we observe the first four results, they have the same performance. I.e., the precision, recall, f1 score & accuracy for the model did not change. Hence I selected the model with the least number of features, which is the first one with just “Mean Texture”. From the fifth iteration onwards, as a feature kept getting added, the accuracy & f1 score kept decreasing. Therefore we did not select any of the remaining models as our best.

Below are the performance metrics of the model for all the feature combinations:

1. Single feature:

- {'precision': 0.9714285714285714, 'recall': 0.8717948717948718, 'accuracy': 0.85, 'f1': 0.9189189189189189, 'confusion matrix': [[0, 1], [5, 34]], 'features': 'Mean Radius'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture'}
- {'precision': 0.9428571428571428, 'recall': 0.868421052631579, 'accuracy': 0.825, 'f1': 0.904109589041096, 'confusion matrix': [[0, 2], [5, 33]], 'features': 'Mean Perimeter'}
- {'precision': 0.9142857142857143, 'recall': 0.8888888888888888, 'accuracy': 0.825, 'f1': 0.9014084507042254, 'confusion matrix': [[1, 3], [4, 32]], 'features': 'Mean Area'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Smoothness'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Compactness'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Concavity'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Concave Points'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Fractal Dimension'}
- {'precision': 0.9142857142857143, 'recall': 0.8888888888888888, 'accuracy': 0.825, 'f1': 0.9014084507042254, 'confusion matrix': [[1, 3], [4, 32]], 'features': 'SE Perimeter'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'SE Texture'}
- {'precision': 0.9142857142857143, 'recall': 0.8888888888888888, 'accuracy': 0.825, 'f1': 0.9014084507042254, 'confusion matrix': [[1, 3], [4, 32]], 'features': 'SE Area'}

2. Two Features:

- {'precision': 0.9142857142857143, 'recall': 0.8648648648648649, 'accuracy': 0.8, 'f1': 0.8888888888888889, 'confusion matrix': [[0, 3], [5, 32]], 'features': 'Mean Texture, Mean Radius'}
- {'precision': 0.9142857142857143, 'recall': 0.8648648648648649, 'accuracy': 0.8, 'f1': 0.8888888888888889, 'confusion matrix': [[0, 3], [5, 32]], 'features': 'Mean Texture, Mean Perimeter'}
- {'precision': 0.8857142857142857, 'recall': 0.8857142857142857, 'accuracy': 0.8, 'f1': 0.8857142857142857, 'confusion matrix': [[1, 4], [4, 31]], 'features': 'Mean Texture, Mean Area'}
- **{'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness'}**
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Compactness'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Concavity'}
- {'precision': 0.9714285714285714, 'recall': 0.8717948717948718, 'accuracy': 0.85, 'f1': 0.9189189189189189, 'confusion matrix': [[0, 1], [5, 34]], 'features': 'Mean Texture, Mean Concave Points'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Fractal Dimension'}
- {'precision': 0.9142857142857143, 'recall': 0.8888888888888888, 'accuracy': 0.825, 'f1': 0.9014084507042254, 'confusion matrix': [[1, 3], [4, 32]], 'features': 'Mean Texture, SE Perimeter'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, SE Texture'}
- {'precision': 0.9142857142857143, 'recall': 0.8888888888888888, 'accuracy': 0.825, 'f1': 0.9014084507042254, 'confusion matrix': [[1, 3], [4, 32]], 'features': 'Mean Texture, SE Area'}

3. **Three Features:**

- {'precision': 0.9142857142857143, 'recall': 0.8648648648648649, 'accuracy': 0.8, 'f1': 0.8888888888888889, 'confusion matrix': [[0, 3], [5, 32]], 'features': 'Mean Texture, Mean Smoothness, Mean Radius'}
- {'precision': 0.9142857142857143, 'recall': 0.8648648648648649, 'accuracy': 0.8, 'f1': 0.8888888888888889, 'confusion matrix': [[0, 3], [5, 32]], 'features': 'Mean Texture, Mean Smoothness, Mean Perimeter'}
- {'precision': 0.8857142857142857, 'recall': 0.8857142857142857, 'accuracy': 0.8, 'f1': 0.8857142857142857, 'confusion matrix': [[1, 4], [4, 31]], 'features': 'Mean Texture, Mean Smoothness, Mean Area'}

- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness, Mean Concavity'}
- {'precision': 0.9714285714285714, 'recall': 0.8717948717948718, 'accuracy': 0.85, 'f1': 0.9189189189189189, 'confusion matrix': [[0, 1], [5, 34]], 'features': 'Mean Texture, Mean Smoothness, Mean Concave Points'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness, Mean Fractal Dimension'}
- {'precision': 0.9142857142857143, 'recall': 0.8888888888888888, 'accuracy': 0.825, 'f1': 0.9014084507042254, 'confusion matrix': [[1, 3], [4, 32]], 'features': 'Mean Texture, Mean Smoothness, SE Perimeter'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness, SE Texture'}
- {'precision': 0.9142857142857143, 'recall': 0.8888888888888888, 'accuracy': 0.825, 'f1': 0.9014084507042254, 'confusion matrix': [[1, 3], [4, 32]], 'features': 'Mean Texture, Mean Smoothness, SE Area'}

4. Four feature:

- {'precision': 0.9428571428571428, 'recall': 0.868421052631579, 'accuracy': 0.825, 'f1': 0.904109589041096, 'confusion matrix': [[0, 2], [5, 33]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Radius'}
- {'precision': 0.9142857142857143, 'recall': 0.8648648648648649, 'accuracy': 0.8, 'f1': 0.8888888888888889, 'confusion matrix': [[0, 3], [5, 32]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Perimeter'}
- {'precision': 0.8857142857142857, 'recall': 0.8857142857142857, 'accuracy': 0.8, 'f1': 0.8857142857142857, 'confusion matrix': [[1, 4], [4, 31]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Area'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concave Points'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Fractal Dimension'}
- {'precision': 0.9142857142857143, 'recall': 0.8888888888888888, 'accuracy': 0.825, 'f1': 0.9014084507042254, 'confusion matrix': [[1, 3], [4, 32]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, SE Perimeter'}

- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, SE Texture'}
- {'precision': 0.9142857142857143, 'recall': 0.8888888888888888, 'accuracy': 0.825, 'f1': 0.9014084507042254, 'confusion matrix': [[1, 3], [4, 32]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, SE Area'}

5. Five Features:

- {'precision': 0.8571428571428571, 'recall': 0.8571428571428571, 'accuracy': 0.75, 'f1': 0.8571428571428571, 'confusion matrix': [[0, 5], [5, 30]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Radius'}
- {'precision': 0.8285714285714286, 'recall': 0.8529411764705882, 'accuracy': 0.725, 'f1': 0.8405797101449276, 'confusion matrix': [[0, 6], [5, 29]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Perimeter'}
- {'precision': 0.8285714285714286, 'recall': 0.8787878787878788, 'accuracy': 0.75, 'f1': 0.8529411764705883, 'confusion matrix': [[1, 6], [4, 29]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Area'}
- {'precision': 0.9142857142857143, 'recall': 0.8648648648648649, 'accuracy': 0.8, 'f1': 0.8888888888888889, 'confusion matrix': [[0, 3], [5, 32]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Concave Points'}
- **{'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension'}**
- {'precision': 0.8857142857142857, 'recall': 0.8857142857142857, 'accuracy': 0.8, 'f1': 0.8857142857142857, 'confusion matrix': [[1, 4], [4, 31]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, SE Perimeter'}
- {'precision': 1.0, 'recall': 0.875, 'accuracy': 0.875, 'f1': 0.9333333333333333, 'confusion matrix': [[0, 0], [5, 35]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, SE Texture'}
- {'precision': 0.9428571428571428, 'recall': 0.8918918918918919, 'accuracy': 0.85, 'f1': 0.9166666666666667, 'confusion matrix': [[1, 2], [4, 33]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, SE Area'}

6. Six Features:

- {'precision': 0.8285714285714286, 'recall': 0.8787878787878788, 'accuracy': 0.75, 'f1': 0.8529411764705883, 'confusion matrix': [[1, 6], [4, 29]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Radius'}

- {'precision': 0.8285714285714286, 'recall': 0.8787878787878788, 'accuracy': 0.75, 'f1': 0.8529411764705883, 'confusion matrix': [[1, 6], [4, 29]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Perimeter'}
- {'precision': 0.8, 'recall': 0.875, 'accuracy': 0.725, 'f1': 0.8358208955223881, 'confusion matrix': [[1, 7], [4, 28]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Area'}
- {'precision': **0.9428571428571428**, 'recall': **0.868421052631579**, 'accuracy': **0.825**, 'f1': **0.904109589041096**, 'confusion matrix': [[0, 2], [5, 33]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points'}
- {'precision': 0.8, 'recall': 0.875, 'accuracy': 0.725, 'f1': 0.8358208955223881, 'confusion matrix': [[1, 7], [4, 28]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, SE Perimeter'}
- {'precision': 0.9428571428571428, 'recall': 0.868421052631579, 'accuracy': 0.825, 'f1': 0.904109589041096, 'confusion matrix': [[0, 2], [5, 33]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, SE Texture'}
- {'precision': 0.8571428571428571, 'recall': 0.8823529411764706, 'accuracy': 0.775, 'f1': 0.8695652173913043, 'confusion matrix': [[1, 5], [4, 30]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, SE Area'}

7. Seven feature:

- {'precision': 0.7714285714285715, 'recall': 0.8709677419354839, 'accuracy': 0.7, 'f1': 0.8181818181818182, 'confusion matrix': [[1, 8], [4, 27]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, Mean Radius'}
- {'precision': 0.7714285714285715, 'recall': 0.8709677419354839, 'accuracy': 0.7, 'f1': 0.8181818181818182, 'confusion matrix': [[1, 8], [4, 27]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, Mean Perimeter'}
- {'precision': 0.7714285714285715, 'recall': 0.8709677419354839, 'accuracy': 0.7, 'f1': 0.8181818181818182, 'confusion matrix': [[1, 8], [4, 27]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, Mean Area'}
- {'precision': 0.7714285714285715, 'recall': 0.9, 'accuracy': 0.725, 'f1': 0.8307692307692307, 'confusion matrix': [[2, 8], [3, 27]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Perimeter'}
- {'precision': **0.9142857142857143**, 'recall': **0.8648648648648649**, 'accuracy': **0.8**, 'f1': **0.8888888888888889**, 'confusion matrix': [[0, 3], [5, 32]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness,

Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture'}

- {'precision': 0.8285714285714286, 'recall': 0.8787878787878788, 'accuracy': 0.75, 'f1': 0.8529411764705883, 'confusion matrix': [[1, 6], [4, 29]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Area'}

8. Eight Features:

- {'precision': 0.7714285714285715, 'recall': 0.8709677419354839, 'accuracy': 0.7, 'f1': 0.8181818181818182, 'confusion matrix': [[1, 8], [4, 27]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, Mean Radius'}
- {'precision': 0.7428571428571429, 'recall': 0.8666666666666667, 'accuracy': 0.675, 'f1': 0.8, 'confusion matrix': [[1, 9], [4, 26]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, Mean Perimeter'}
- {'precision': 0.7714285714285715, 'recall': 0.8709677419354839, 'accuracy': 0.7, 'f1': 0.8181818181818182, 'confusion matrix': [[1, 8], [4, 27]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, Mean Area'}
- {'precision': **0.7714285714285715**, 'recall': **0.9**, 'accuracy': **0.725**, 'f1': **0.8307692307692307**, 'confusion matrix': [[2, 8], [3, 27]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter'}
- {'precision': 0.7714285714285715, 'recall': 0.8709677419354839, 'accuracy': 0.7, 'f1': 0.8181818181818182, 'confusion matrix': [[1, 8], [4, 27]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Area'}

9. Nine Features:

- {'precision': 0.6857142857142857, 'recall': 0.8888888888888888, 'accuracy': 0.65, 'f1': 0.7741935483870968, 'confusion matrix': [[2, 11], [3, 24]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter, Mean Radius'}
- {'precision': 0.6857142857142857, 'recall': 0.8888888888888888, 'accuracy': 0.65, 'f1': 0.7741935483870968, 'confusion matrix': [[2, 11], [3, 24]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter, Mean Perimeter'}
- {'precision': 0.6857142857142857, 'recall': 0.8888888888888888, 'accuracy': 0.65, 'f1': 0.7741935483870968, 'confusion matrix': [[2, 11], [3, 24]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter, Mean Area'}

- {'precision': 0.7142857142857143, 'recall': 0.8928571428571429, 'accuracy': 0.675, 'f1': 0.7936507936507937, 'confusion matrix': [[2, 10], [3, 25]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter, SE Area'}

10. Ten feature:

- {'precision': 0.6571428571428571, 'recall': 0.8846153846153846, 'accuracy': 0.625, 'f1': 0.7540983606557377, 'confusion matrix': [[2, 12], [3, 23]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter, SE Area, Mean Radius'}
- {'precision': 0.6571428571428571, 'recall': 0.8846153846153846, 'accuracy': 0.625, 'f1': 0.7540983606557377, 'confusion matrix': [[2, 12], [3, 23]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter, SE Area, Mean Perimeter'}
- {'precision': 0.6571428571428571, 'recall': 0.8846153846153846, 'accuracy': 0.625, 'f1': 0.7540983606557377, 'confusion matrix': [[2, 12], [3, 23]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter, SE Area, Mean Area'}

11. Eleven Features:

- {'precision': 0.6571428571428571, 'recall': 0.8846153846153846, 'accuracy': 0.625, 'f1': 0.7540983606557377, 'confusion matrix': [[2, 12], [3, 23]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter, SE Area, Mean Radius, Mean Perimeter'}
- {'precision': 0.6571428571428571, 'recall': 0.8846153846153846, 'accuracy': 0.625, 'f1': 0.7540983606557377, 'confusion matrix': [[2, 12], [3, 23]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter, SE Area, Mean Radius, Mean Area'}

12. All Features:

- {'precision': 0.6571428571428571, 'recall': 0.92, 'accuracy': 0.65, 'f1': 0.7666666666666667, 'confusion matrix': [[3, 12], [2, 23]], 'features': 'Mean Texture, Mean Smoothness, Mean Compactness, Mean Concavity, Mean Fractal Dimension, Mean Concave Points, SE Texture, SE Perimeter, SE Area, Mean Radius, Mean Perimeter, Mean Area'}
- c. Compare the performance of the full model built using all the features in (3a) with the resultant accuracies of the full model using the selected features (3b). Which set of features performed better?
Evaluate performance of your models using a metric discussed in class

(such as the confusion matrix). You may also use graphs to explain your observations.

Performance Metric	Value 3b (Forward Splitwise Best model)	Value 3a (12 Features Logistic Regression)
Confusion Matrix	[[0, 0], [5, 35]]	[[2, 3], [4, 31]]
Precision	<u>1.0</u>	0.9117647058823529
Recall	<u>0.875</u>	0.8857142857142857
F1 Score	<u>0.9333333333333333</u>	0.8985507246376812
Accuracy	<u>0.875</u>	0.825
Best Features	<u>Mean Texture</u>	Mean radius, Mean texture, Mean perimeter, Mean area, Mean smoothness, Mean compactness, Mean concavity, Mean concave points, Mean fractal dimension, SE perimeter, SE texture, SE area

My Observation:

Based on the results from the above table, the Accuracy & F1 score for the forward splitwise logistic regression produces the best results compared to the model with all the 12 features. Both the models have very close precision & recall values. However, for evaluating the performance of the whole model, we are comparing the accuracy & f1 score. The forward splitwise model has F1 Score & Accuracy of 0.934 & 0.875 respectively. Whereas the 12 feature model has F1 Score of 0.898 & Accuracy of 0.825. Though the values are very close, we chose the model that gave us better numbers, the one with a single feature (Mean Texture). Hence this is our best performing model on comparing both the models.

4. Experimenting with regularization and Cost function. (40 points)

a. Regularization and Feature Scaling: (20 points)

i. For the best performing model in Q.3 (Model from 3c), does regularization improve the performance?

I performed regularization on the best performing model from the previous question. I have assigned the lambda value set as 1000 and set the learning rate to 0.02 to curve the cost values. The cost, gradient & confusion matrix value comparison before and after regularization have been consolidated in the table below. I calculated the gradient descent on the training dataset and calculated the cost, predictions & performance evaluation on the testing dataset.

Performance Metric	Before Regularization	After Regularization
Cost	0.4367553149305674	0.4384293929862341
W (theta1)	[-0.00465592]	[-0.00021082]
C (theta0)	1.0151135371042423	0.9990450831758252

Confusion Matrix	[[0, 0], [5, 35]]	[[0, 5], [0, 35]]
Precision	1.0	0.875
Recall	0.875	1.0
F1 Score	0.9333333333333333	0.9333333333333333
Accuracy	0.875	0.875

My Observation: There is close to no change (~ 0.01) in the cost of the model before and after regularization. When it comes to the performance of the model, there is no change in the F1 Score & the Accuracy of the model. Both the models perform in the same wavelengths. So I have chosen the original model without regularization as the best model.

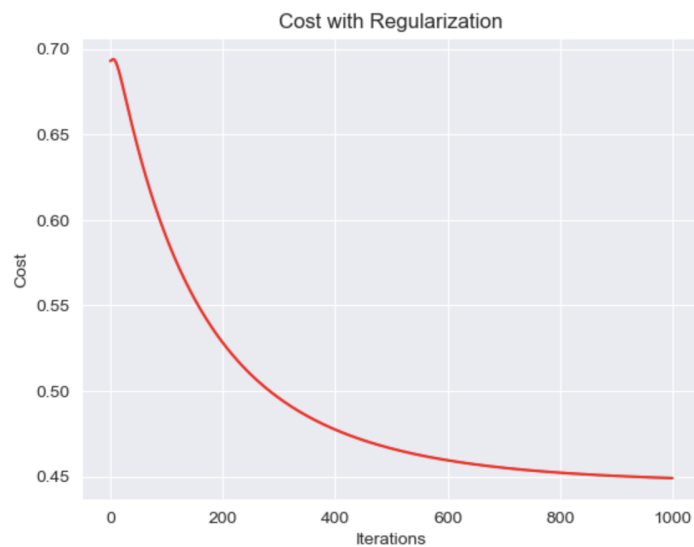


Fig 5: Graph shows the cost function being stabilised through multiple iterations

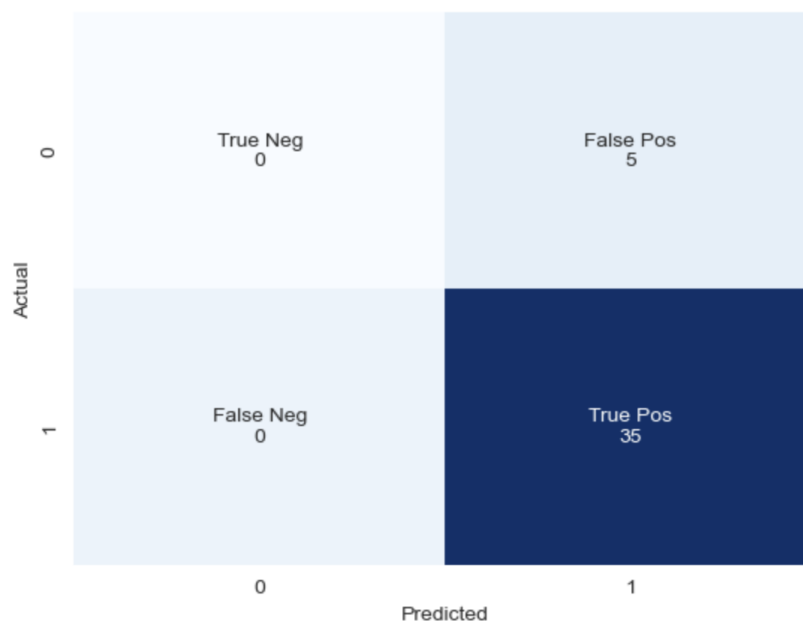


Fig 6: Confusion Matrix for regularized cost

ii. Does Feature Scaling improve the performance for the model in Q 3c?

Evaluate performance for each case using a metric discussed in class (such as confusion matrix). You may also use graphs to explain your observations.

The best model from 3c was built with feature scaling the formula $x - \text{Mean}(x) / \text{Standard Deviation}(x)$. In this question, I have reconstructed the model without using feature scaling. During this process, the values of Cost, gradient & confusion matrix are as follows:

Performance Metric	With Feature Scaling	Without Feature Scaling
Cost	0.4367553149305674	0.4129486299887901
W (theta1)	[-0.00465592]	[0.04577992]
C (theta0)	1.0151135371042423	0.004703688258118817
Confusion Matrix	[[0, 0], [5, 35]]	[[0, 5], [0, 35]]
Precision	1.0	0.875
Recall	0.875	1.0
F1 Score	0.9333333333333333	0.9333333333333333
Accuracy	0.875	0.875

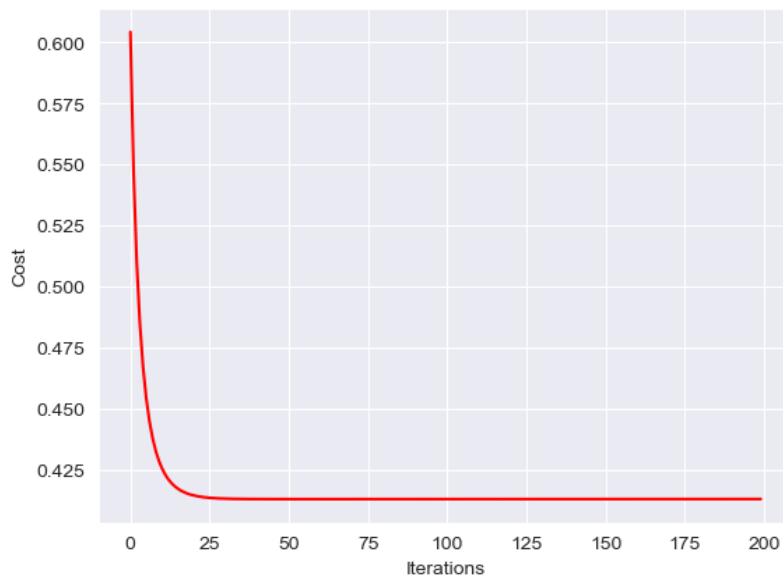


Fig 7: Cost over multiple iterations without feature scaling

My Observation: There is close to no change in the cost of the models with & without feature scaling. In fact, the cost of the model increased by ~ 0.2 with feature scaling.

The performance of the model with & without feature scaling did not change. The precision, recall, accuracy & f1 scores are the same. Based on this, it is clear to state that feature scaling had no impact on the best model's performance.

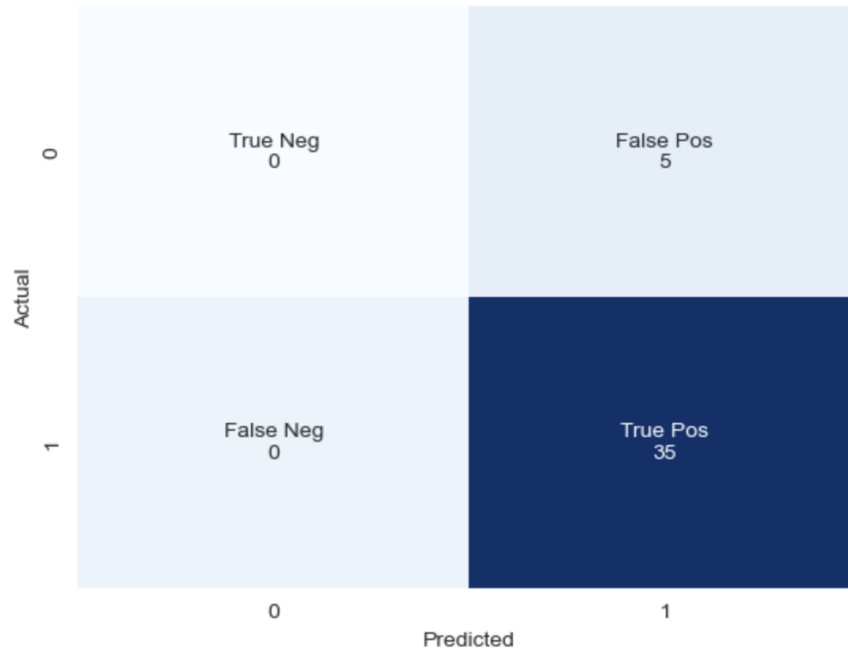


Fig 8: Confusion Matrix for Model without feature scaling

b. Cost Function: (20 points)

- i. Keeping the best model after the experiments from Q.4a, design a Logistic Regression model to predict breast cancer recurrence (Outcome) by changing the cost function to the following (Mean Squared Error).

I implemented logistic regression on the dataset, split the data into training and test buckets and calculated the theta values (Intercepts and coefficient) using Gradient Descent formula. However, here I changed the cost formula to MSE. The W, C, Cost & performance values are as follows:

Metric	Old Cost Function	New Cost Function (MSE)
Cost	0.4367553149305674	0.4010533577408915
W (theta1)	[-0.00465592]	[-0.00465592]
C (theta0)	1.0151135371042423	1.0151135370224025

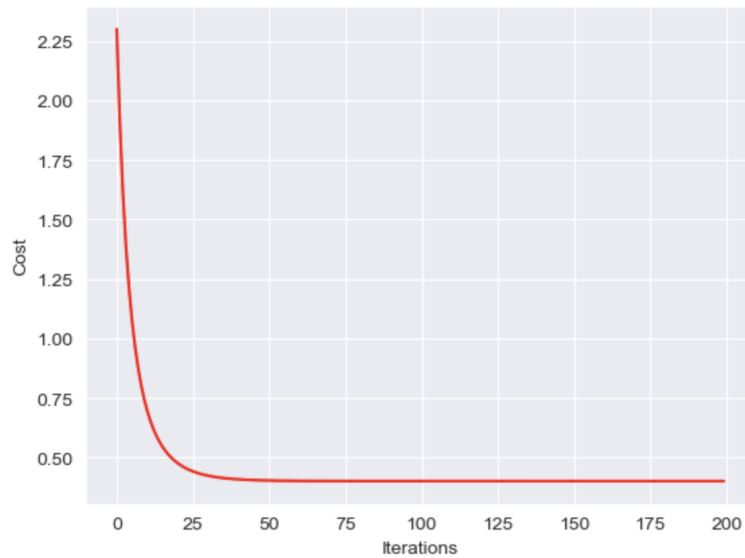


Fig 9: Graph shows the cost function being stabilised through multiple iterations

My Observation: On comparing both the metrics, it is clear that there is no difference in the cost derived using the MSE & Logistic equation cost formulas. The new cost function increased the cost by ~ 0.03 which is not a large increase.

- ii. **Compare the performance of both the models (4.b.i and 4.a.). Do they give the same solution with a difference in cost function? Evaluate performance for each case using a metric discussed in class (such as confusion matrix). You may also use graphs to explain your observations**

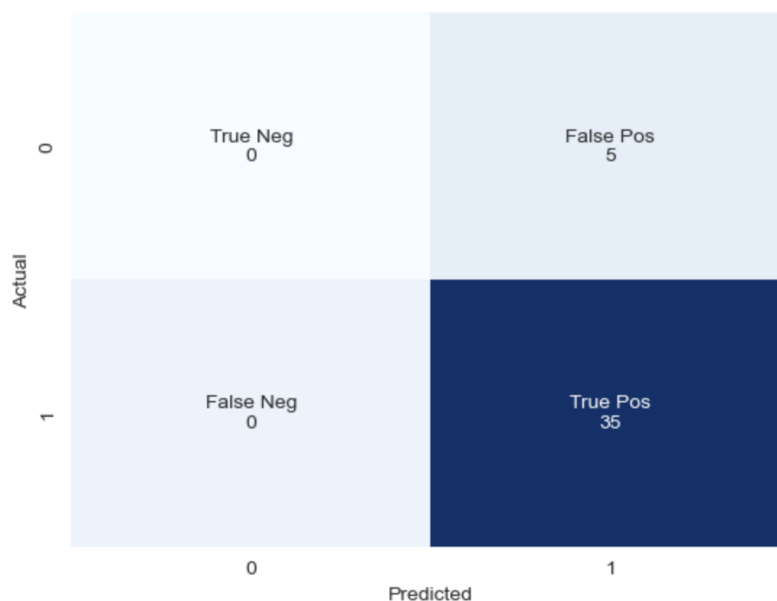


Fig 10: Confusion Matrix for Model derived for the new cost function

Performance Metric	Old Performance Values	New Performance (MSE)
Confusion Matrix	[[0, 0], [5, 35]]	[[0, 5], [0, 35]]
Precision	1.0	0.875
Recall	0.875	1.0
F1 Score	0.9333333333333333	0.9333333333333333
Accuracy	0.875	0.875

My Observation: I compared the performance based on the confusion matrix, the precision, recall, f1 score and accuracy, both the models have the same results. Based on this I can confirm that both models have the same performance (with precision and recall switching places) and a slight difference in the cost is observed.

When it comes to the overall model selection, I would go ahead with the logistic cost function model. Because When we change the cost function to MSE and replace $h\theta(x)$ with sigmoid function, the formula for gradient descent should also change. But here we used the old gradient formula itself and it is not efficient to compare these two models. We would have to derive a new formula for the gradient and then re-run the whole model.